

Paradigmas de la Programación – Segundo Parcial

7 de Mayo de 2026

Apellido y Nombre: _____

Ej. 1

Ej. 2

Ej. 3

Ej. 4

Ejercicio 1 Niveles de visibilidad en orientación a objetos

25 pt

Definiciones

La visibilidad de un miembro (atributo o método) de una clase determina desde qué partes del programa puede ser accedido. Los tres niveles fundamentales son:

Público (public): el miembro es accesible desde cualquier parte del programa, sin restricciones. Cualquier clase, en cualquier paquete o módulo, puede leerlo y modificarlo.

Protegido (protected): el miembro es accesible desde la propia clase, desde sus subclases y, en Java, también desde otras clases dentro del mismo paquete.

Privado (private): el miembro es accesible únicamente desde el interior de la propia clase donde fue declarado. Ninguna subclase ni clase externa puede acceder a él directamente, incluso si pertenece al mismo paquete.

Código de referencia

Java

Archivo Animal.java (paquete bio)

```
1 package bio;
2 public class Animal {
3     public String nombre;
4     protected int edad;
5     private double peso;
6
7     public double getPeso() {
8         return peso;
9     }
10 }
```

Archivo Perro.java (paquete bio)

```
1 package bio;
2 public class Perro extends Animal {
3     public void info() {
4         System.out.println(nombre); // a1
5         System.out.println(edad);   // a2
6         System.out.println(peso);   // a3
7     }
8 }
```

Archivo Gato.java (paquete mascota)

```
1 package mascota;
2 import bio.Animal;
3 public class Gato extends Animal {
4     public void info() {
5         System.out.println(nombre); // a4
6         System.out.println(edad);   // a5
7         System.out.println(peso);   // a6
8     }
9 }
```

Archivo Main.java (paquete mascota)

```
1 package mascota;
2 import bio.Animal;
3 public class Main {
4     public static void main(String[]
5         args){
6         Animal a = new Animal();
7         System.out.println(a.nombre); //
8             a7
9         System.out.println(a.edad);   //
10            a8
11         System.out.println(a.getPeso());
12            // a9
13     }
14 }
```

Verdadero o Falso. Indique **V** o **F** para cada afirmación.

#	Afirmación	V / F
<i>Java</i>		
1	La línea a1 (<code>System.out.println(nombre)</code>) da un error de compilación.	
2	La línea a2 (<code>System.out.println(edad)</code>) da un error de compilación.	
3	La línea a3 (<code>System.out.println(peso)</code>) da un error de compilación.	
4	Si sustituimos la línea a3 por <code>System.out.println(getPeso())</code> , el código de <code>Perro</code> compila sin problemas.	
5	La línea a4 (<code>System.out.println(nombre)</code>) en <code>Gato</code> produce un error.	
6	La línea a5 (<code>System.out.println(edad)</code>) en <code>Gato</code> produce un error.	
7	La línea a6 (<code>System.out.println(peso)</code>) en <code>Gato</code> produce un error.	
8	Si sustituimos la línea a6 por <code>System.out.println(getPeso())</code> , el código de <code>Gato</code> compila sin problemas.	
9	La línea a7 (<code>System.out.println(a.nombre)</code>) en <code>Main</code> produce un error.	
10	La línea a8 (<code>System.out.println(a.edad)</code>) en <code>Main</code> produce un error.	
11	La línea a9 (<code>System.out.println(a.getPeso())</code>) en <code>Main</code> produce un error de compilación.	
12	Si sustituimos la línea a8 por <code>System.out.println(a.getPeso())</code> , el código de <code>Main</code> compila sin problemas.	
13	Si moviéramos la clase <code>Main</code> al paquete <code>bio</code> , la línea a8 (<code>System.out.println(a.edad)</code>) dejaría de producir un error.	
14	Si moviéramos la clase <code>Gato</code> al paquete <code>bio</code> , la línea a5 (<code>System.out.println(edad)</code>) dejaría de producir un error.	

C++

```

1 class Vehiculo {
2 public:
3     std::string marca;
4 protected:
5     int anio;
6 private:
7     double precio;
8     double descuento(double pct) {
9         return precio * pct;
10    }
11 };
12
13 class Auto : public Vehiculo {
14 public:
15     void mostrar() {
16         std::cout << marca // b1
17                     << std::endl;
18         std::cout << anio // b2

```

```

19         << std::endl;
20         std::cout << precio // b3
21                 << std::endl;
22         std::cout // b4
23                 << descuento(0.1)
24                 << std::endl;
25     }
26 };
27
28 int main() {
29     Auto a;
30     std::cout << a.marca // b5
31             << std::endl;
32     std::cout << a.anio // b6
33             << std::endl;
34     std::cout << a.precio // b7
35             << std::endl;
36 }

```

#	Afirmación	V / F
15	La línea b1 (<code>std::cout << marca</code>) dentro de <code>Auto::mostrar</code> produce un error de compilación.	
16	La línea b2 (<code>std::cout << anio</code>) dentro de <code>Auto::mostrar</code> produce un error de compilación.	
17	La línea b3 (<code>std::cout << precio</code>) dentro de <code>Auto::mostrar</code> produce un error de compilación.	
18	La línea b4 (<code>std::cout << descuento(0.1)</code>) dentro de <code>Auto::mostrar</code> produce un error de compilación.	
19	La línea b5 (<code>std::cout << a.marca</code>) en <code>main</code> produce un error de compilación.	
20	La línea b6 (<code>std::cout << a.anio</code>) en <code>main</code> produce un error de compilación.	
21	La línea b7 (<code>std::cout << a.precio</code>) en <code>main</code> produce un error de compilación.	
22	Si sustituimos la línea b3 por <code>std::cout << Vehiculo::precio << std::endl;</code> , el código de <code>Auto::mostrar</code> compila sin problemas.	
23	Si declaramos la herencia de <code>Auto</code> como <code>private</code> (<code>class Auto : private Vehiculo</code>), la línea b1 (<code>std::cout << marca</code>) dentro de <code>Auto::mostrar</code> sigue compilando sin problemas.	
24	Si declaramos la herencia de <code>Auto</code> como <code>private</code> (<code>class Auto : private Vehiculo</code>), la línea b5 (<code>std::cout << a.marca</code>) en <code>main</code> produce un error de compilación.	
25	Si añadimos en <code>Vehiculo</code> el método público <code>double getDescuento(double p){return descuento(p);}</code> , y sustituimos la línea b4 por <code>std::cout << getDescuento(0.1) << std::endl;</code> , el código de <code>Auto::mostrar</code> compila sin problemas.	

Ejercicio 2 Interfaces

25 pt

Definiciones.

Función virtual pura / clase abstracta (C++). Una función virtual tipo `f() = 0` es **puramente virtual**: obliga a toda subclase concreta a redefinirla. La clase que la contiene es **abstracta** y no puede instanciarse. Una subclase que no implementa todas las funciones puras heredadas sigue siendo abstracta. Las demás funciones de la clase pueden tener implementación concreta.

Interfaz (Java). Una `interface` declara métodos implícitamente `public abstract`. Desde Java 8 puede incluir métodos `default` con implementación. Si dos interfaces implementadas por la misma clase definen un `default` con la misma firma, el compilador exige que la clase resuelva la ambigüedad sobreescribiéndolo explícitamente.

Código de referencia.

C++

```
1 class Figura {
2 public:
3     virtual double area() = 0;           //
4     virtual std::string tipo() = 0;     //
5     virtual void imprimir() {           //
6         std::cout << tipo()
7             << ": " << area();
8     }
9     virtual ~Figura() = default;
10 };
11
12 class Rect : public Figura {
13 public:
14     double b, h;
15     Rect(double b, double h)
16         : b(b), h(h) {}
17     double area() override {
18         return b * h;
19     }
20     std::string tipo() override {
21         return "Rect";
22     }
23 };
24
25 int main() {
26     // Figura f;
27     Figura* p = new Rect(3, 4);
28     p->imprimir();
29     std::cout << p->area();
30     delete p;
31 }
```

Java

```
1 interface Figura {
2     double area();
3     default void imprimir() {
4         System.out.println(
5             area());
6     }
7 }
8
9 interface Nombrada {
10     default void imprimir() {
11         System.out.println("obj");
12     }
13 }
14
15 class Rect implements
16     Figura, Nombrada {
17     double b, h;
18     Rect(double b, double h) {
19         this.b = b; this.h = h;
20     }
21     public double area() {
22         return b * h;
23     }
24     // imprimir() no definido
25 }
26
27 class Main {
28     public static void main(
29         String[] args) {
30         // new Figura();
31         Figura f = new Rect(3, 4);
32         f.imprimir();
33         System.out.println(
34             f.area());
35     }
36 }
```

Verdadero o Falso. Indique V o F.

#	Afirmación	V / F
1	El código c6 (Figura f;) produce un error de compilación.	
2	El código c7 (Figura* p = new Rect(3, 4)) produce un error de compilación.	
3	Si eliminamos c2 (la función virtual std::string tipo() = 0) de Figura, el código c6 (Figura f;) deja de producir un error de compilación.	
4	El código j7 (new Figura()) produce un error de compilación.	
5	El código j4 (class Rect implements Figura, Nombrada) produce un error de compilación.	
6	Si añadimos en Rect el método public void imprimir() { Figura.super.imprimir(); }, el código j4 deja de producir un error de compilación.	

Ejercicio 3 Herencia y subtipado

25 pt

Definiciones.

Interfaz de un objeto en Smalltalk. La **interfaz** de un objeto es el conjunto de mensajes a los que ese objeto sabe responder. En Smalltalk no existe una declaración explícita de tipo ni de interfaz: la interfaz de un objeto queda determinada por los métodos definidos en su clase y en todas las superclases de esa clase.

Contención de interfaces y subtipado. Se dice que la interfaz de la clase B **contiene** a la interfaz de la clase A si todo mensaje al que responde un objeto de A también es respondido por un objeto de B , es decir, $interfaz(A) \subseteq interfaz(B)$. Cuando esto ocurre, un objeto de B puede usarse en cualquier contexto que espere un objeto de A : decimos que B es un **subtipo** de A . En Smalltalk la herencia garantiza automáticamente la contención: si B hereda de A , la interfaz de B contiene a la de A (porque B hereda todos los métodos de A y puede agregar más). Sin embargo, la contención *no requiere* herencia: dos clases sin relación jerárquica pueden tener interfaces que se contienen si definen los mismos mensajes (**duck typing**).

Código de referencia.

```

1 Object subclass: #Animal
2   instanceVariableNames: 'nombre'
3   classVariableNames: ''
4   poolDictionaries: ''
5   category: ''.
6
7 Animal >> nombre [
8   ^ nombre
9 ]
10
11 Animal >> nombre: unNombre [
12   nombre := unNombre
13 ]
14
15 Animal >> hablar [
16   ^ 'Silencio'
17 ]
18
19 Animal >> describir [
20   ^ nombre , ' dice: ' , self hablar
21 ]
22
23 Animal subclass: #Perro
24   instanceVariableNames: ''
25   classVariableNames: ''
26   poolDictionaries: ''
27   category: ''.
28
29 Perro >> hablar [
30   ^ 'Guau'

```

<pre> 31] 32 33 Animal subclass: #Gato "s7" 34 instanceVariableNames: 'vidas' 35 classVariableNames: '' 36 poolDictionaries: '' 37 category: ''. 38 39 Gato >> vidas ["s8" 40 ^ vidas 41] 42 43 Gato >> vidas: n ["s9" 44 vidas := n 45] 46 47 Gato >> hablar ["s10" 48 ^ 'Miau' 49] 50 51 Object subclass: #Robot "s11" 52 instanceVariableNames: 'nombre' 53 classVariableNames: '' 54 poolDictionaries: '' 55 category: ''. 56 57 Robot >> nombre ["s12" 58 ^ nombre 59] 60 61 Robot >> nombre: n ["s13" </pre>	<pre> 62 nombre := n 63] 64 65 Robot >> hablar ["s14" 66 ^ 'Bip bip' 67] 68 69 "--- Código cliente ---" 70 71 a p g r 72 a := Animal new. "s15" 73 a nombre: 'Animal'. 74 75 p := Perro new. "s16" 76 p nombre: 'Firulaís'. 77 78 g := Gato new. "s17" 79 g nombre: 'Michi'. 80 g vidas: 9. 81 82 r := Robot new. "s18" 83 r nombre: 'R2D2'. 84 85 Transcript show: (a describir). "s19" 86 Transcript show: (p describir). "s20" 87 Transcript show: (g describir). "s21" 88 Transcript show: (r describir). "s22" 89 Transcript show: (r nombre). "s23" 90 Transcript show: (g vidas printString). "s24" </pre>
---	---

Verdadero o Falso. Indique V o F.

#	Afirmación	V / F
1	La interfaz de Perro contiene a la interfaz de Animal porque Perro hereda de Animal.	
2	La interfaz de Animal contiene a la interfaz de Perro porque Perro hereda de Animal.	
3	Perro hereda el método describir (s4) de Animal.	
4	La línea s19 (a describir) imprime 'Animal dice: Silencio'.	
5	La línea s21 (g describir) imprime 'Michi dice: Miau'.	
6	La línea s22 (r describir) imprime 'R2D2 dice: Bip bip'	
7	Un objeto de clase Perro puede usarse en cualquier contexto que espere un Animal.	
8	Un objeto de clase Robot puede usarse en cualquier contexto que espere un Animal.	
9	Un objeto de clase Robot puede usarse en cualquier contexto que espere un Gato.	

Definiciones.

Encapsulamiento en el modelo de actores. El estado interno de un actor debe ser accesible y modificable *únicamente* por el propio actor. Toda interacción entre actores ocurre exclusivamente a través del envío de **mensajes inmutables**.

Código de referencia.

```
1 import akka.actor.{Actor, ActorSystem, Props, ActorRef}
2
3 // --- Mensajes ---
4 case class Depositar(monto: Double)
5 case object ObtenerSaldo
6
7 // --- Actor cuenta: almacena el saldo ---
8 class CuentaActor extends Actor {
9     var saldo: Double = 0.0 // linea A
10
11     def receive: Receive = {
12         case Depositar(monto) =>
13             saldo += monto // linea B
14             println(s"Saldo actualizado: $saldo")
15
16         case ObtenerSaldo =>
17             sender() ! saldo // linea C
18     }
19 }
20
21 // --- Actor cliente: opera sobre la cuenta ---
22 class ClienteActor(cuenta: ActorRef) extends Actor {
23
24     def receive: Receive = {
25         case "iniciar" =>
26             cuenta ! Depositar(100.0) // linea D
27
28         case saldo: Double =>
29             println(s"[Cliente] Saldo recibido: $saldo")
30     }
31 }
32
33 // --- Actor auditor: modifica el estado de cuenta directamente ---
34 class AuditorActor(cuentaRef: CuentaActor) extends Actor { // linea F
35     def receive: Receive = {
36         case "auditar" =>
37             cuentaRef.saldo = 0.0 // linea G
38             println("[Auditor] Saldo reseteado")
39
40         case "leer" =>
41             println(s"[Auditor] Saldo: ${cuentaRef.saldo}") // linea H
42     }
43 }
44
45 object Main extends App {
46     val system = ActorSystem("banco")
47     val cuenta = system.actorOf(Props[CuentaActor], "cuenta")
48     val cliente = system.actorOf(
49         Props(new ClienteActor(cuenta)), "cliente")
50 }
```

```

50
51 cliente ! "iniciar" // linea I
52 cuenta ! ObtenerSaldo // linea J
53 Thread.sleep(1000)
54 system.terminate()
55 }

```

Verdadero o Falso. Indique **V** o **F** para cada afirmación.

#	Afirmación	V / F
A	La línea G (<code>cuentaRef.saldo = 0.0</code>) viola el encapsulamiento del actor <code>CuentaActor</code> .	
B	La línea H (<code>cuentaRef.saldo</code>) viola el encapsulamiento del actor <code>CuentaActor</code> .	
C	Si en la línea G tuviéramos <code>cuentaRef ! Depositar(-cuentaRef.saldo)</code> , estaríamos violando el encapsulamiento del actor <code>CuentaActor</code> .	
D	La línea C (<code>sender() ! saldo</code>) viola el encapsulamiento del actor emisor del mensaje.	
E	La línea B (<code>saldo += monto</code>) dentro de <code>receive</code> es una sección crítica.	
F	La línea D (<code>cuenta ! Depositar(100.0)</code>) viola el encapsulamiento del actor <code>CuentaActor</code> .	
G	La línea I (<code>cliente ! iniciar</code>), que envía un <code>String</code> , viola el encapsulamiento de los actores involucrados.	
H	La línea J (<code>cuenta ! ObtenerSaldo</code>) no puede producir una condición de carrera.	